DPPL-LAPAKITA-01

DESKRIPSI PERANCANGAN PERANGKAT LUNAK

LAPAKITA

Aplikasi Marketplace Berbasis Web dan Android

Mata Kuliah Implementasi dan Pengujian Perangkat Lunak

CAK3BAB3 - IF-48-PJJ-01

Dosen Pengampu: Eko Darwiyanto, S.T., M.T.

Dipersiapkan oleh Kelompok Lapakita

M. Raditya Faturrahman (1304221050)
Bima Evri Okta Ramadhan (1304221077)
Regina Dea Ananda (1304221084)
Eko Widodo (1304221037)

Program Studi S1 Informatika

Fakultas Informatika, Universitas Telkom

2026

Daftar Perubahan

Revisi	Deskripsi
A	Rancangan awal berisi arsitektur dan realisasi use case Checkout serta Proses Pesanan
B	Penambahan algoritma validasi transisi status dan pembatalan otomatis
C	Penyesuaian dengan SKPL revisi D: modul notifikasi, wishlist, invoice, pembatalan oleh pembeli, mode libur, stok menipis, dan penurunan produk
D	
E	

INDEX	-	A	B	C	D	E	F	G
TGL		20/10/2026	27/10/2026	06/11/2026				
Ditulis oleh		Kel. Lapakita	Kel. Lapakita	Kel. Lapakita				
Diperiksa oleh		M. Raditya F.	M. Raditya F.	M. Raditya F.				
Disetujui oleh								

Daftar Halaman Perubahan

Halaman	Revisi	Halaman	Revisi
20	B	21	B
8-9	C	20-30	C

Daftar Isi

Daftar Isi 3
Daftar Tabel 3
Daftar Gambar 3
1. Pendahuluan 5
    1.1 Tujuan Penulisan Dokumen 5
    1.2 Lingkup Masalah 5
    1.3 Definisi dan Istilah 5
    1.4 Referensi 6
2. Perancangan Global 7
    2.1 Rencana Lingkungan Implementasi 7
    2.2 Deskripsi Arsitektur Perangkat Lunak 7
3. Perancangan Rinci 10
    3.1 Realisasi Use Case 10
        3.1.1 Use Case #1 Checkout 11
        3.1.2 Use Case #2 Proses Pesanan 15
        3.1.3 Use Case #3 Proses Notifikasi Pembayaran 18
    3.2 Diagram Kelas Keseluruhan 20
    3.3 Perancangan Detil Kelas 21
    3.4 Perancangan Algoritma dan Query 24
4. Matriks Kerunutan 30

Daftar Tabel

Tabel 1.1 Definisi dan istilah 5
Tabel 2.1 Rencana lingkungan implementasi 7
Tabel 2.2 Daftar komponen 8
Tabel 3.1 Daftar use case yang direalisasikan 10
Tabel 3.2 Objek UI layar Checkout 12
Tabel 3.3 Objek perancangan use case Checkout 13
Tabel 3.4 Objek UI layar Pesanan Toko 15
Tabel 3.5 Objek perancangan use case Proses Pesanan 16
Tabel 3.6 Objek perancangan use case Proses Notifikasi Pembayaran 18
Tabel 3.7 Perancangan detil kelas 21
Tabel 3.8 Pilihan urutan hasil pencarian 28
Tabel 4.1 Matriks kerunutan kebutuhan 30

Daftar Gambar

Gambar 2.1 Diagram komponen 8
Gambar 3.1 Rancangan layar Checkout (Android) 12
Gambar 3.2 Sequence diagram use case Checkout 13
Gambar 3.3 Rancangan layar Pesanan Toko (web) 15
Gambar 3.4 Sequence diagram use case Proses Pesanan 17
Gambar 3.5 Sequence diagram use case Proses Notifikasi Pembayaran 19
Gambar 3.6 Diagram kelas keseluruhan 20

1. Pendahuluan

1.1 Tujuan Penulisan Dokumen

Dokumen ini menurunkan kebutuhan pada SKPL-LAPAKITA-01 menjadi rancangan yang bisa langsung dikerjakan, mulai dari pembagian komponen, rancangan layar, kelas perancangan, sampai algoritma untuk bagian yang perhitungannya tidak sepele. Tujuannya supaya empat anggota bisa bekerja paralel di backend, web, dan Android tanpa saling menunggu atau menafsirkan kebutuhan secara berbeda.

Selain untuk tim, dokumen ini dipakai dosen pengampu untuk menilai kesesuaian rancangan dengan kebutuhan, dan nanti menjadi dasar penyusunan kasus uji serta pengukuran OO Metrics.

1.2 Lingkup Masalah

Lapakita adalah aplikasi marketplace berbasis web dan Android dengan satu backend bersama. Sistem menangani pendaftaran, login, logout, profil, dan pengelolaan kata sandi tiga peran pengguna, pengelolaan toko dan produk termasuk mode libur dan stok menipis, wishlist, keranjang belanja lintas toko, checkout yang memecah pesanan per toko, pembayaran melalui Midtrans mode sandbox, pembatalan, pemrosesan, dan pelacakan pesanan, invoice, ulasan beserta balasannya, notifikasi dalam aplikasi, serta fungsi administrasi platform.

1.3 Definisi dan Istilah

Tabel 1.1 Definisi dan istilah

Istilah	Keterangan
Boundary	Kelas yang menangani tampilan dan masukan pengguna, baik halaman web maupun layar Android
Control	Kelas yang mengatur alur proses. Pada rancangan ini dibagi menjadi Controller (penerima permintaan) dan Service (logika bisnis)
Entity	Kelas yang mewakili data dan dipetakan ke tabel basis data
Repository	Antarmuka akses data berbasis Spring Data JPA
DTO	Data Transfer Object, objek sederhana untuk memindahkan data antar lapisan
Pessimistic lock	Penguncian baris data selama transaksi berjalan supaya tidak diubah proses lain
Scheduler	Komponen yang menjalankan tugas secara berkala tanpa dipicu pengguna
Endpoint	Alamat REST API yang dapat dipanggil klien
Snap token	Token sekali pakai dari Midtrans untuk membuka halaman pembayaran satu transaksi

Istilah	Keterangan
Webhook	Pemanggilan HTTP dari Midtrans ke server Lapakita setiap kali status pembayaran berubah
Signature key	Hash SHA-512 pada notifikasi Midtrans untuk membuktikan keaslian pesan
Idempoten	Sifat proses yang memberi hasil sama walaupun dijalankan berkali-kali dengan masukan yang sama

1.4 Referensi

Kelompok Lapakita. (2026). SKPL-LAPAKITA-01, Spesifikasi Kebutuhan Perangkat Lunak Lapakita.

Kelompok Lapakita. (2026). Perencanaan Konstruksi Perangkat Lunak Lapakita (Revisi Tugas 1).

Larman, C. (2004). Applying UML and Patterns, Edisi Ketiga. Upper Saddle River: Prentice Hall.

Tim Teaching SE-IMPTEST. (2026). Template DPPL Berorientasi Objek. Program Studi S1 Informatika, Universitas Telkom.

Midtrans. (2026). Snap Integration Guide dan HTTP(S) Notification. Diakses dari https://docs.midtrans.com pada Oktober 2026.

2. Perancangan Global

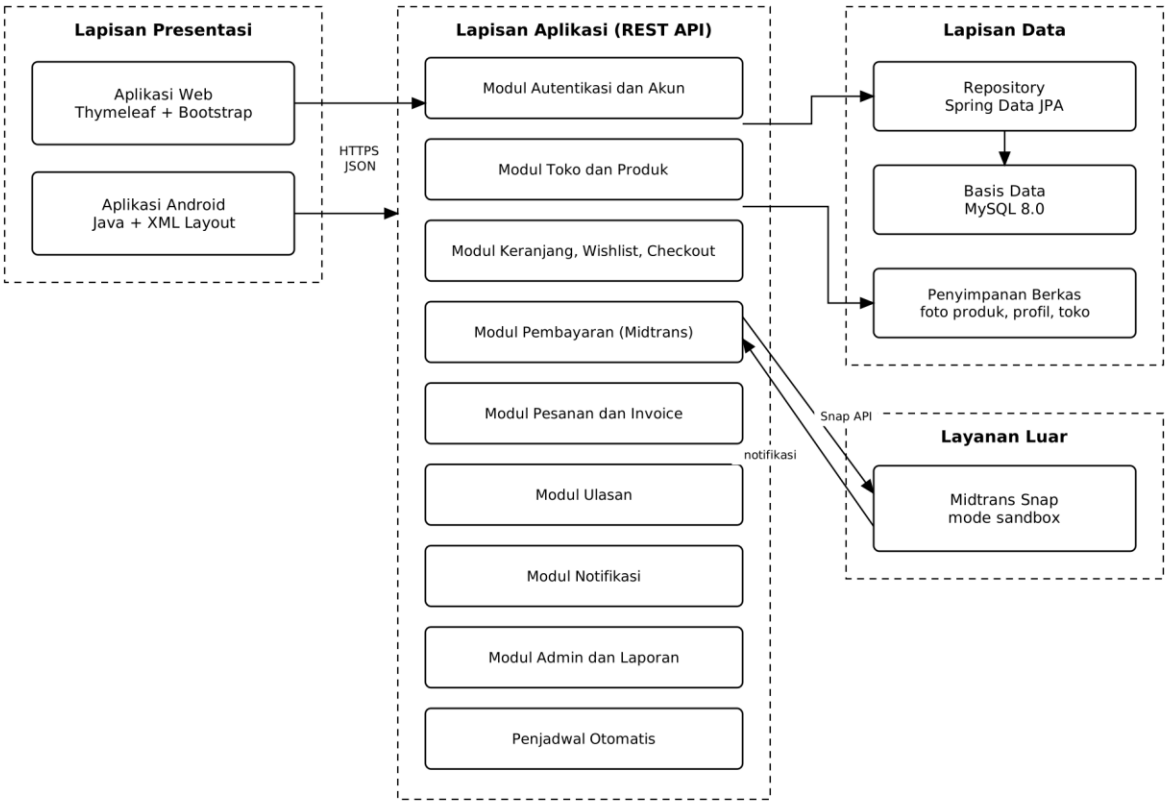
2.1 Rencana Lingkungan Implementasi

Lapakita memakai arsitektur tiga lapis. Lapisan presentasi berisi aplikasi web dan Android, lapisan aplikasi berisi seluruh logika bisnis yang dibungkus sebagai REST API, dan lapisan data berisi repository, basis data, serta folder penyimpanan berkas. Karena fitur web dan Android dibuat sama lengkapnya, memusatkan logika di backend adalah satu-satunya cara agar keduanya tidak menghasilkan perhitungan yang berbeda, misalnya pada ongkos kirim atau total bayar.

Tabel 2.1 Rencana lingkungan implementasi

Bagian	Teknologi
Bahasa	Java 17 untuk backend dan Android
Backend	Spring Boot 3, Spring Web, Spring Data JPA, Spring Security dengan token JWT, Spring Scheduling
Basis data	MySQL 8.0 dengan InnoDB supaya mendukung transaksi dan penguncian baris
Web	Thymeleaf dan Bootstrap 5, memanggil REST API dari sisi peramban
Android	Android Studio, layout XML, Retrofit untuk memanggil REST API
Payment gateway	Midtrans Snap mode sandbox melalui library midtrans-java
Tunneling	ngrok untuk membuka endpoint notifikasi ke internet selama pengembangan
Pengujian	JUnit 5 dan Mockito untuk unit test, Postman untuk pengujian endpoint, simulator pembayaran Midtrans
Repositori kode	GitHub, satu repository dengan tiga folder: backend, web, android

2.2 Deskripsi Arsitektur Perangkat Lunak



Gambar 2.1 Diagram komponen sistem Lapakita

Tabel 2.2 Daftar komponen

No	Komponen	Tanggung Jawab
1	Aplikasi Web	Halaman untuk ketiga peran. Hanya menampilkan data dan meneruskan masukan ke API
2	Aplikasi Android	Layar untuk ketiga peran dengan fungsi yang sama dengan web
3	Modul Autentikasi dan Akun	Registrasi, login, pembuatan token, pembatasan akses per peran, profil, dan atur ulang kata sandi
4	Modul Toko dan Produk	Pengajuan toko, profil toko dan mode libur, pengelolaan produk dan stok menipis, pencarian dan halaman toko
5	Modul Keranjang, Wishlist, Checkout	Keranjang lintas toko, wishlist, perhitungan ongkos kirim, catatan untuk penjual, pemecahan pesanan, reservasi stok
6	Modul Pembayaran (Midtrans)	Membuat snap token, menerima notifikasi, memverifikasi signature, dan sinkronisasi status
7	Modul Pesanan dan Invoice	Perubahan status pesanan dengan validasi transisi, pembatalan oleh pembeli, nomor resi, konfirmasi diterima, pembuatan invoice PDF
8	Modul Ulasan	Pemberian ulasan, balasan penjual, dan perhitungan ulang rating toko
9	Modul Notifikasi	Membuat, menampilkan, dan menandai notifikasi dalam aplikasi, serta mengirim email

No	Komponen	Tanggung Jawab
10	Modul Admin dan Laporan	Verifikasi toko, kategori, penurunan produk, pemblokiran, dashboard, laporan penjualan toko
11	Penjadwal Otomatis	Cadangan pembatalan bila notifikasi expire tidak diterima, penyelesaian pesanan lewat 3 hari dikirim, penghapusan notifikasi lebih dari 90 hari, dan pengakhiran mode libur
12	Repository dan Basis Data	Penyimpanan seluruh data relasional
13	Penyimpanan Berkas	Foto produk, foto profil, dan foto toko
14	Midtrans Snap (layanan luar)	Halaman pembayaran, pemrosesan VA, QRIS, e-wallet, dan pengirim notifikasi

Komponen pada lapisan presentasi tidak pernah mengakses repository secara langsung. Aturan ini dijaga supaya seluruh logika bisnis bisa diuji dengan unit test tanpa menjalankan antarmuka apa pun.

3. Perancangan Rinci

3.1 Realisasi Use Case

Seluruh use case pada SKPL direalisasikan. Tiga di antaranya diuraikan sampai sequence diagram karena paling rumit. Checkout melibatkan perhitungan, pemecahan pesanan, dan penguncian stok, Proses Pesanan mewakili sisi penjual dan memakai validasi transisi status, sedangkan Proses Notifikasi Pembayaran adalah satu-satunya titik masuk perubahan status pembayaran dari Midtrans.

Tabel 3.1 Daftar use case yang direalisasikan

No	Nama Use Case	Deskripsi
#1	Checkout	Mengubah produk terpilih di keranjang menjadi satu transaksi dan beberapa pesanan per toko
#2	Proses Pesanan	Penjual menerima, mengemas, dan mengirim pesanan dengan nomor resi, atau menolaknya
#3	Proses Notifikasi Pembayaran	Server menerima notifikasi Midtrans, memverifikasinya, lalu mengubah status pesanan
#4	Autentikasi	Registrasi, login, logout, ganti password, dan lupa password
#5	Cari Produk	Pencarian dengan kata kunci, filter, pengurutan, dan halaman toko
#6	Kelola Keranjang dan Alamat	Mengelola item keranjang dan alamat pengiriman
#7	Bayar Pesanan	Pembeli membayar melalui halaman Snap Midtrans
#8	Lacak dan Konfirmasi Pesanan	Pembeli melihat status, mengunduh invoice, dan menandai pesanan diterima
#9	Beri Ulasan	Pembeli memberi rating dan komentar, penjual membalasnya
#10	Buka Toko dan Kelola Produk	Pengajuan toko, profil toko, mode libur, pengelolaan produk, dan stok menipis
#11	Laporan Toko	Rekap penjualan toko per periode
#12	Fungsi Admin	Monitoring transaksi, verifikasi toko, kategori, blokir akun, penurunan produk, dashboard
#13	Kelola Profil	Mengubah nama, nomor HP, dan foto profil
#14	Lihat Notifikasi	Menampilkan notifikasi dan menandainya sudah dibaca
#15	Kelola Wishlist	Menyimpan dan menghapus produk favorit
#16	Batalkan Pesanan	Pembeli membatalkan transaksi yang belum dibayar
#17	Lihat Bantuan	Halaman statis bantuan, syarat dan ketentuan, kebijakan privasi

3.1.1 Use Case #1 Checkout

3.1.1.1 Use Case Scenario

Nama Use Case	Checkout
Aktor	Pembeli
Pre-Condition	Pembeli sudah login, memiliki alamat, dan ada produk terpilih di keranjang
Use Case Description	Produk terpilih diubah menjadi satu transaksi. Produk dari toko berbeda dipecah menjadi pesanan terpisah, ongkos kirim dihitung per pesanan, dan stok dikurangi
Primary Flow	1. Pembeli memilih produk di keranjang lalu menekan Checkout 2. Sistem menampilkan produk yang dikelompokkan per toko beserta alamat utama 3. Pembeli memilih kurir untuk setiap toko 4. Sistem menghitung ongkos kirim dan total bayar 5. Pembeli menekan Buat Pesanan 6. Sistem mengunci dan memeriksa ulang stok 7. Sistem membuat transaksi dan satu pesanan per toko 8. Sistem mengurangi stok dan mengosongkan item keranjang terkait 9. Sistem meminta snap token ke Midtrans dengan batas waktu 24 jam 10. Sistem membuka halaman pembayaran Midtrans
Alternative Flow	2a. Toko salah satu produk tidak aktif, produk ditandai tidak tersedia 6a. Stok tidak mencukupi, seluruh proses dibatalkan dan produk bermasalah ditampilkan 7a. Terjadi galat saat menyimpan, seluruh perubahan dibatalkan (rollback) 9a. Midtrans tidak merespons, transaksi tetap tersimpan dan pembeli dapat meminta ulang halaman pembayaran dari menu Pesanan Saya
Post-Condition	Transaksi berstatus Menunggu Pembayaran tersimpan bersama pesanannya, stok sudah terpotong

3.1.1.2 UI Design dan Deskripsi Objek UI



Gambar 3.1 Rancangan layar Checkout (Android)

Layar web untuk use case yang sama memakai objek yang sama dengan susunan dua kolom, yaitu daftar pesanan per toko di kiri dan ringkasan pembayaran di kanan. Karena objek dan perilakunya sama, tabel berikut berlaku untuk kedua platform.

Tabel 3.2 Objek UI layar Checkout

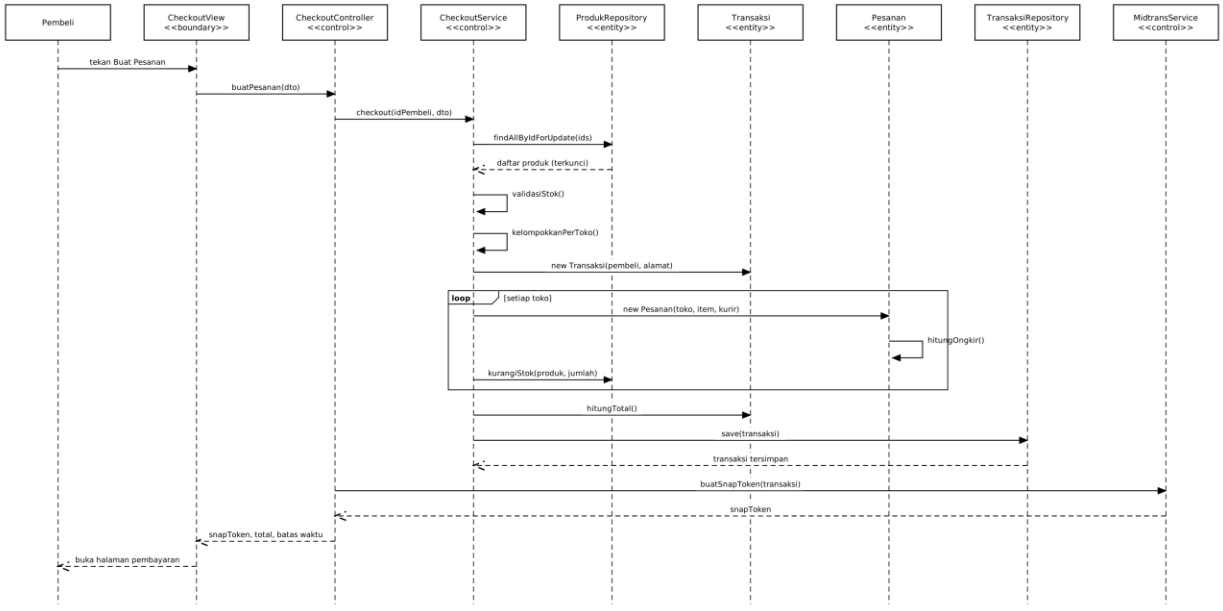
Id_Objek	Jenis	Label	Keterangan
LBL1	Label	Alamat Pengiriman	Menampilkan alamat utama pembeli
BTN1	Button	UBAH ALAMAT	Membuka daftar alamat. Alamat yang dipilih memicu hitung ulang ongkos kirim
CMB1, CMB2	ComboBox	Kurir	Satu ComboBox untuk setiap toko, berisi Ekonomi, Reguler, dan Kilat beserta ongkosnya
LBL2	Label	Ringkasan	Subtotal, ongkos kirim, biaya layanan, dan total bayar. Diperbarui setiap kurir berubah
BTN2	Button	BUAT PESANAN	Memanggil buatPesanan() pada CheckoutController. Nonaktif sampai semua toko sudah dipilih kurirnya

3.1.1.3 Identifikasi Object dan Tipe Kelas

Tabel 3.3 Objek perancangan use case Checkout

No	Nama Object	Tipe Kelas	Tanggung Jawab
1	CheckoutView	Boundary	Menampilkan rincian checkout dan menerima pilihan kurir
2	CheckoutController	Control	Menerima permintaan dari klien dan meneruskannya ke service
3	CheckoutService	Control	Logika checkout: validasi stok, pemecahan per toko, ongkos kirim, transaksi database
4	OngkirCalculator	Control	Menghitung ongkos kirim dari kurir dan berat (BR-04)
5	Transaksi	Entity	Satu kali checkout beserta total dan batas bayar
6	Pesanan	Entity	Bagian transaksi dari satu toko
7	ItemPesanan	Entity	Produk, jumlah, dan harga saat beli
8	Produk	Entity	Sumber harga, berat, dan stok
9	ProdukRepository	Entity	Membaca produk dengan penguncian baris
10	TransaksiRepository	Entity	Menyimpan transaksi beserta pesanannya
11	CheckoutRequestDTO	Entity	Daftar id item keranjang, id alamat, dan kurir per toko

3.1.1.4 Sequence Diagram



Gambar 3.2 Sequence diagram use case Checkout

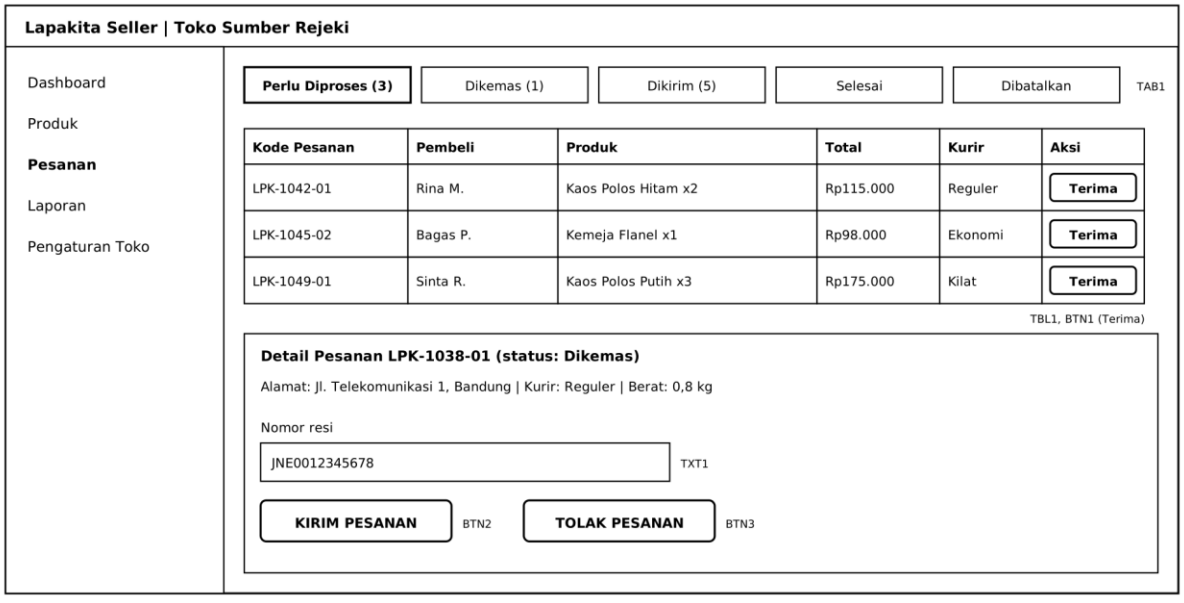
Pesan findAllByIdForUpdate mengunci baris produk sampai transaksi database selesai. Bila dua pembeli checkout produk terakhir bersamaan, pembeli kedua menunggu sampai pembeli pertama selesai, lalu mendapati stok sudah nol dan checkout-nya ditolak. Inilah cara rancangan memenuhi NFR-03. Permintaan snap token sengaja dilakukan setelah transaksi tersimpan dan di luar transaksi database, supaya waktu tunggu jaringan ke Midtrans tidak ikut menahan kunci stok.

3.1.2 Use Case #2 Proses Pesanan

3.1.2.1 Use Case Scenario

Nama Use Case	Proses Pesanan
Aktor	Penjual
Pre-Condition	Penjual sudah login, toko aktif, dan ada pesanan berstatus Perlu Diproses
Use Case Description	Penjual menerima pesanan, mengemas, lalu mengirim dengan mengisi nomor resi. Penjual juga dapat menolak pesanan
Primary Flow	1. Penjual membuka menu Pesanan Toko 2. Sistem menampilkan pesanan per status 3. Penjual menekan Terima pada pesanan Perlu Diproses 4. Sistem memvalidasi transisi lalu mengubah status menjadi Dikemas 5. Penjual mengisi nomor resi dan menekan Kirim Pesanan 6. Sistem menyimpan resi dan mengubah status menjadi Dikirim
Alternative Flow	3a. Penjual menekan Tolak Pesanan, status menjadi Dibatalkan dan stok dikembalikan 5a. Nomor resi kosong atau kurang dari 8 karakter, sistem menolak 5b. Penjual menekan Kirim pada pesanan yang belum diterima, sistem menolak karena transisi tidak sah
Post-Condition	Pesanan berstatus Dikirim dengan nomor resi, atau Dibatalkan dengan stok dikembalikan

3.1.2.2 UI Design dan Deskripsi Objek UI



Gambar 3.3 Rancangan layar Pesanan Toko (web)

Tabel 3.4 Objek UI layar Pesanan Toko

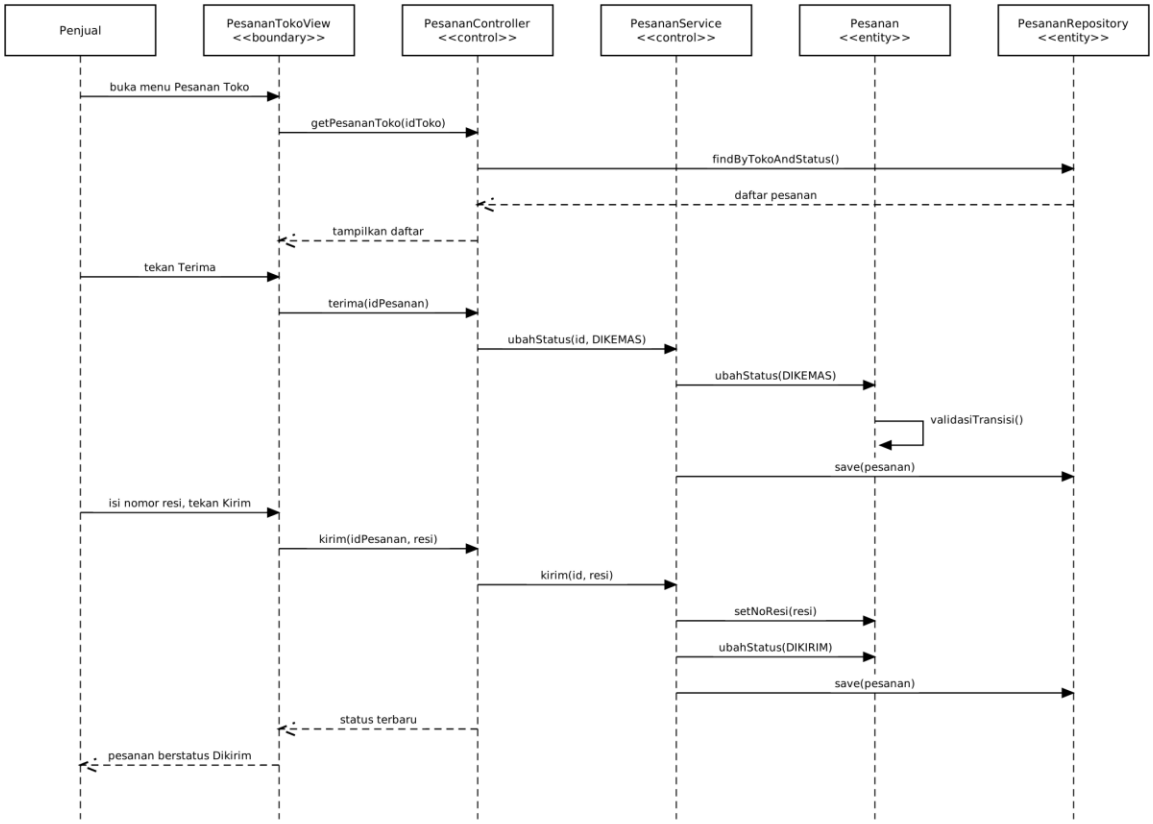
Id_Objek	Jenis	Label	Keterangan
TAB1	Tab	Status pesanan	Lima tab status dengan jumlah pesanan. Memfilter isi TBL1
TBL1	Table	Daftar pesanan	Kode pesanan, pembeli, produk, total, dan kurir
BTN1	Button	Terima	Muncul pada tab Perlu Diproses, memanggil terima() pada PesananController
TXT1	TextBox	Nomor resi	Wajib diisi minimal 8 karakter sebelum pesanan dikirim
BTN2	Button	KIRIM PESANAN	Memanggil kirim() dengan nomor resi. Aktif hanya pada status Dikemas
BTN3	Button	TOLAK PESANAN	Meminta konfirmasi dan alasan, lalu memanggil tolak()

3.1.2.3 Identifikasi Object Baru dan Tipe Kelas

Tabel 3.5 Objek perancangan use case Proses Pesanan

No	Nama Object	Tipe Kelas	Tanggung Jawab
1	PesananTokoView	Boundary	Menampilkan pesanan per status dan form nomor resi
2	PesananController	Control	Menerima permintaan terima, kirim, dan tolak
3	PesananService	Control	Memeriksa kepemilikan toko, menjalankan perubahan status, mengembalikan stok saat ditolak
4	StatusPesanan	Entity	Enum tujuh status beserta daftar transisi yang sah
5	Pesanan	Entity	Sudah diidentifikasi pada use case #1, ditambah method ubahStatus() dan setNoResi()
6	PesananRepository	Entity	Membaca pesanan per toko dan status, menyimpan perubahan

3.1.2.4 Sequence Diagram



Gambar 3.4 Sequence diagram use case Proses Pesanan

3.1.3 Use Case #3 Proses Notifikasi Pembayaran

3.1.3.1 Use Case Scenario

Nama Use Case	Proses Notifikasi Pembayaran
Aktor	Midtrans (aktor sistem)
Pre-Condition	Transaksi sudah memiliki snap token dan pembeli sudah melakukan atau membatalkan pembayaran di halaman Midtrans
Use Case Description	Midtrans mengirim notifikasi HTTP POST setiap kali status pembayaran berubah. Server memverifikasi keaslian notifikasi, mencocokkan nominal, lalu mengubah status pesanan sesuai status pembayaran
Primary Flow	1. Midtrans mengirim notifikasi berisi order_id, status_code, gross_amount, transaction_status, dan signature_key 2. Sistem menghitung ulang signature dan mencocokkannya 3. Sistem mencari transaksi berdasarkan order_id 4. Sistem memastikan transaksi belum berstatus final 5. Sistem mencocokkan gross_amount dengan total bayar 6. Bila status settlement atau capture, seluruh pesanan menjadi Perlu Diproses 7. Sistem menyimpan perubahan dan membalas HTTP 200
Alternative Flow	2a. Signature tidak cocok, sistem membalas HTTP 403 dan tidak mengubah apa pun 4a. Transaksi sudah final (notifikasi ganda), sistem langsung membalas HTTP 200 tanpa memproses ulang 5a. Nominal tidak cocok, notifikasi ditolak dan dicatat untuk diperiksa admin 6a. Status expire, cancel, atau deny, seluruh pesanan dibatalkan dan stok dikembalikan 6b. Status pending, tidak ada perubahan
Post-Condition	Status pesanan sesuai dengan status pembayaran di Midtrans, dan notifikasi yang sama tidak pernah diproses dua kali

3.1.3.2 UI Design dan Deskripsi Objek UI

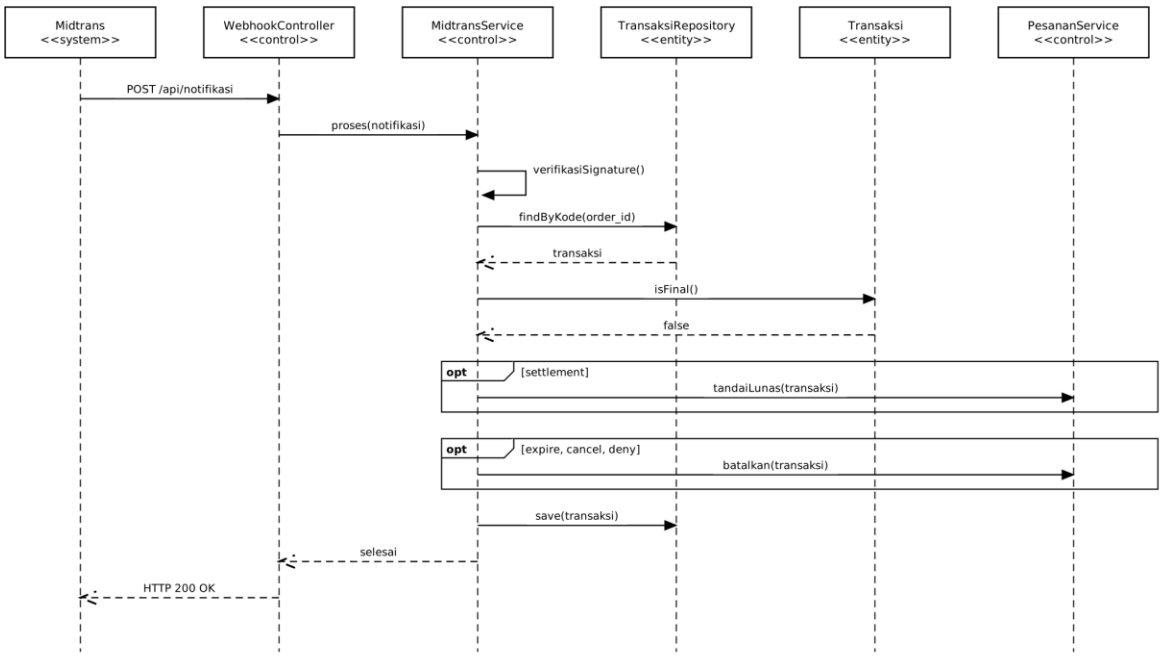
Use case ini tidak memiliki antarmuka pengguna karena dipicu langsung oleh server Midtrans. Batas sistemnya berupa endpoint POST /api/notifikasi/midtrans yang menerima data JSON. Hasil use case ini baru terlihat oleh pengguna sebagai perubahan status pada layar Pesanan Saya milik pembeli dan layar Pesanan Toko milik penjual.

3.1.3.3 Identifikasi Object Baru dan Tipe Kelas

Tabel 3.6 Objek perancangan use case Proses Notifikasi Pembayaran

No	Nama Object	Tipe Kelas	Tanggung Jawab
1	WebhookController	Boundary / Control	Endpoint penerima notifikasi. Tidak memerlukan login karena keasliannya dibuktikan dengan signature
2	MidtransService	Control	Membuat snap token, memverifikasi signature, memetakan status Midtrans ke status pesanan, menyinkronkan status
3	NotifikasiMidtransDTO	Entity	Isi notifikasi: order_id, status_code, gross_amount, transaction_status, signature_key, payment_type
4	Pembayaran	Entity	Menyimpan order id, snap token, metode, status, dan waktu bayar
5	Transaksi	Entity	Sudah diidentifikasi pada use case #1, ditambah method isFinal()
6	PesananService	Control	Sudah diidentifikasi pada use case #2, ditambah tandaiLunas() dan batalkan()

3.1.3.4 Sequence Diagram

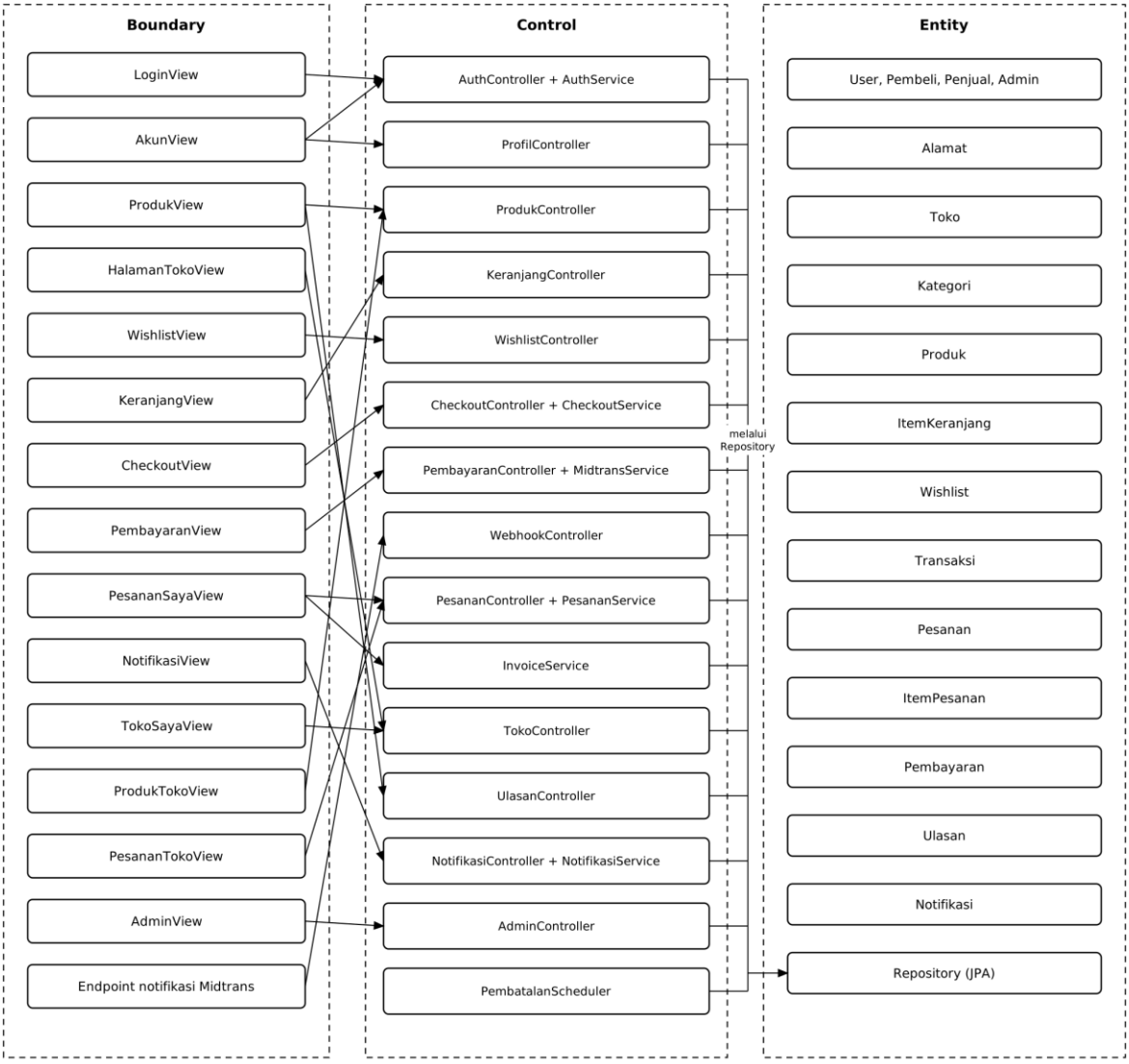


Gambar 3.5 Sequence diagram use case Proses Notifikasi Pembayaran

Midtrans dapat mengirim notifikasi yang sama lebih dari sekali, misalnya bila balasan dari server terlambat. Pemeriksaan isFinal() memastikan notifikasi kedua tidak mengembalikan stok dua kali atau mengubah pesanan yang sudah diproses penjual. Ini memenuhi NFR-09.

3.2 Diagram Kelas Keseluruhan

Kelas perancangan dikelompokkan menjadi boundary, control, dan entity. Pada lapisan control, kelas yang logikanya berat dipecah menjadi Controller dan Service. Controller hanya menerima permintaan dan mengembalikan respons, sedangkan Service berisi aturan bisnis. Pemisahan ini membuat Service bisa diuji dengan Mockito tanpa menjalankan server. Seluruh akses ke entity melewati repository.



Gambar 3.6 Diagram kelas keseluruhan

3.3 Perancangan Detil Kelas

Tabel 3.7 Perancangan detil kelas

ID	Nama Kelas	Atribut	Method
K-01	CheckoutController (Control)	- checkoutService : CheckoutService	+ pratinjau(dto) : RingkasanDTO + buatPesanan(dto) : RingkasanDTO
K-02	CheckoutService (Control)	- produkRepo : ProdukRepository - transaksiRepo : TransaksiRepository - ongkir : OngkirCalculator	+ checkout(idPembeli, dto) : Transaksi - validasiStok(items) : void - kelompokkanPerToko(items) : Map - buatPesanan(toko, items, kurir) : Pesanan
K-03	OngkirCalculator (Control)	- tarifDasar : Map<Kurir, Integer> - tarifPerKg : Map<Kurir, Integer>	+ hitung(kurir, beratGram) : int
K-04	PesananController (Control)	- pesananService : PesananService	+ getPesananToko(idToko, status) : List + terima(id) : PesananDTO + kirim(id, resi) : PesananDTO + tolak(id, alasan) : PesananDTO + konfirmasiDiterima(id) : PesananDTO
K-05	PesananService (Control)	- pesananRepo : PesananRepository - produkRepo : ProdukRepository	+ ubahStatus(id, status) : Pesanan + kirim(id, resi) : Pesanan + tolak(id, alasan) : Pesanan + batalkanOlehPembeli(idTrx, idPembeli) - pastikanPemilik(pesanan, idPenjual)
K-06	MidtransService (Control)	- serverKey : String - snapApi : SnapApi - transaksiRepo - pesananService	+ buatSnapToken(trx) : String + proses(notif) : void - verifikasiSignature(notif) : boolean + cekStatus(kode) : String + batalkanTransaksi(kode) : void + sinkronkanStatus(kode) : void
K-07	PembatalanScheduler (Control)	- transaksiRepo - pesananService - midtransService	+ batalkanKedaluwarsa() : int + selesaikanOtomatis() : int + akhiriLibur() : int + hapusNotifikasiLama() : int
K-08	Transaksi (Entity)	- idTransaksi : Long - kode : String - biayaLayanan : int - batasBayar : LocalDateTime - daftarPesanan : List<Pesanan>	+ hitungTotal() : int + isKedaluwarsa(now) : boolean + tambahPesanan(p : Pesanan) : void
K-09	Pesanan (Entity)	- idPesanan : Long - status : StatusPesanan - kurir : Kurir - ongkir : int - noResi : String - catatan : String - items : List<ItemPesanan>	+ subtotal() : int + total() : int + ubahStatus(baru : StatusPesanan) : void + setNoResi(resi : String) : void
K-10	StatusPesanan (Entity, enum)	MENUNGGU_PEMBAYARAN, PERLU_DIPROSES, DIKEMAS, DIKIRIM, SELESAI, DIBATALKAN	+ bisaPindahKe(tujuan) : boolean
K-11	Produk (Entity)	- idProduk : Long - harga : int - stok : int - batasStok : int - beratGram : int	+ isTersedia(n : int) : boolean + kurangiStok(n : int) : void + tambahStok(n : int) : void + isMenipis() : boolean + turunkan(alasan : String) : void

ID	Nama Kelas	Atribut	Method
		- status : StatusProduk - alasanTurun : String	
K-12	ItemPesanan (Entity)	- jumlah : int - hargaSaatBeli : int	+ subtotal() : int + beratTotal() : int
K-13	Toko (Entity)	- idToko : Long - status : StatusToko - rating : double - liburSampai : LocalDate	+ isAktif() : boolean + isMenerimaPesanan(hari) : boolean + aturLibur(sampai) : void + hitungRating(list) : double
K-14	ProdukRepository (Entity)	(antarmuka JPA)	+ findAllByIdForUpdate(ids) : List<Produk> + cari(keyword, filter, urutan, page) : Page<Produk> + findMenipis(idToko) : List<Produk>
K-15	PesananRepository (Entity)	(antarmuka JPA)	+ findByTokoAndStatus(idToko, status) : List + findDikirimSebelum(batas) : List
K-16	WebhookController (Control)	- midtransService : MidtransService	+ terimaNotifikasi(json) : ResponseEntity
K-17	AuthService (Control)	- userRepo : UserRepository - encoder : PasswordEncoder - tokenStore : TokenStore	+ login(email, pass) : Token + logout(token) : void + gantiPassword(id, lama, baru) + mintaReset(email) : void + resetPassword(token, baru)
K-18	Pembayaran (Entity)	- orderIdGateway : String - snapToken : String - metode : String - status : String - waktuBayar : LocalDateTime	+ perbaruiStatus(s) : void + isLunas() : boolean
K-19	ProfilController (Control)	- userRepo : UserRepository - berkas : FileStorage	+ getProfil(idUser) : ProfilDTO + ubahProfil(idUser, dto) : ProfilDTO + unggahFoto(idUser, file) : String
K-20	NotifikasiService (Control)	- notifRepo : NotifikasiRepository - email : EmailSender	+ kirim(idUser, judul, isi, tautan) : void + daftar(idUser, page) : Page + jumlahBelumDibaca(idUser) : int + tandaiDibaca(id, idUser) : void
K-21	Notifikasi (Entity)	- idNotifikasi : Long - judul : String - isi : String - tautan : String - dibaca : boolean - waktu : LocalDateTime	+ tandaiDibaca() : void
K-22	WishlistController (Control)	- wishlistRepo : WishlistRepository	+ tambah(idUser, idProduk) : void + hapus(idUser, idProduk) : void + daftar(idUser) : List<ProdukDTO>
K-23	InvoiceService (Control)	- pesananRepo : PesananRepository - pdf : PdfGenerator	+ buat(idPesanan, idPembeli) : byte[] - pastikanLunas(pesanan) : void
K-24	Ulasan (Entity)	- rating : int - komentar : String - balasan : String - tanggalBalas : LocalDateTime	+ balas(teks : String) : void + sudahDibalas() : boolean

Nominal uang disimpan sebagai int dalam satuan rupiah, bukan double, karena rupiah tidak memakai pecahan dan tipe double dapat menimbulkan galat pembulatan saat dijumlahkan. Atribut status pada Pesanan hanya bisa diubah lewat ubahStatus() supaya validasi transisi tidak bisa dilewati. Server key Midtrans dibaca dari variabel lingkungan dan tidak ditulis di kode sumber maupun dikirim ke klien.

3.4 Perancangan Algoritma dan Query

Bagian ini memuat proses yang perhitungannya tidak sepele. Operasi simpan dan baca biasa ditangani Spring Data JPA.

Algoritma #1 Perhitungan Ongkos Kirim

Kelas: OngkirCalculator | Operasi: hitung(kurir, beratGram)

Berat total pesanan dijumlahkan dari seluruh item lalu dibulatkan ke atas per kilogram. Kilogram pertama dikenai tarif dasar, kilogram berikutnya dikenai tarif per kg. Pesanan seberat 1.001 gram sudah dihitung dua kilogram. Batas inilah yang akan diuji dengan boundary value analysis.

```
hitung(kurir, beratGram) -> int
    IF beratGram <= 0 THEN
        THROW BeratTidakValidException
    END IF
    beratKg <- CEIL(beratGram / 1000)
    ongkir  <- tarifDasar[kurir] + (beratKg - 1) * tarifPerKg[kurir]
    RETURN ongkir

// tarifDasar : EKONOMI 9000,  REGULER 15000,  KILAT 25000
// tarifPerKg : EKONOMI 4000,  REGULER  6000,  KILAT 10000
```

Contoh: Reguler, 2.300 gram, dibulatkan 3 kg, ongkir = 15.000 + 2 x 6.000 = Rp27.000.

Algoritma #2 Checkout dengan Pemecahan Pesanan dan Penguncian Stok

Kelas: CheckoutService | Operasi: checkout(idPembeli, dto)

```
@Transactional
checkout(idPembeli, dto) -> Transaksi
    items  <- keranjangRepo.findTerpilih(idPembeli, dto.idItems)
    IF items KOSONG THEN THROW KeranjangKosongException

    // kunci baris produk sampai transaksi selesai
    produk <- produkRepo.findAllByIdForUpdate(id produk dari items)
    FOR EACH item IN items
        IF NOT produk[item].isTersedia(item.jumlah) THEN
            // seluruh transaksi di-rollback
            THROW StokTidakCukupException(item)
        END IF
        IF NOT produk[item].toko.isAktif() THEN
            THROW TokoTidakAktifException(item)
        END IF
    END FOR

    trx <- new Transaksi(pembeli, alamat, biayaLayanan = 1000,
                         batasBayar = SEKARANG + 24 jam)
    FOR EACH (toko, itemsToko) IN kelompokkanPerToko(items)
        pesanan <- new Pesanan(toko, dto.kurir[toko])
        FOR EACH item IN itemsToko
            pesanan.tambahItem(produk, item.jumlah, produk.harga)
            produk.kurangiStok(item.jumlah)
```

```
        END FOR
        pesanan.ongkir <- ongkir.hitung(pesanan.kurir, pesanan.beratTotal())
        pesanan.status <- MENUNGGU_PEMBAYARAN
        trx.tambahPesanan(pesanan)
    END FOR

    transaksiRepo.save(trx)
    keranjangRepo.deleteAll(items)
    RETURN trx
```

Seluruh method berjalan dalam satu transaksi database. Pengecualian apa pun di tengah jalan membuat semua perubahan dibatalkan, termasuk stok yang sudah terlanjur dikurangi. Ini memenuhi NFR-05.

Algoritma #3 Validasi Transisi Status

Kelas: StatusPesanan dan Pesanan | Operasi: bisaPindahKe(tujuan), ubahStatus(baru)

```
TRANSISI_SAH = {
    MENUNGGU_PEMBAYARAN : [PERLU_DIPROSES, DIBATALKAN],
    PERLU_DIPROSES      : [DIKEMAS, DIBATALKAN],
    DIKEMAS             : [DIKIRIM],
    DIKIRIM             : [SELESAI],
    SELESAI             : [],
    DIBATALKAN          : []
}

ubahStatus(baru)
    IF NOT TRANSISI_SAH[this.status] BERISI baru THEN
        THROW TransisiTidakSahException(this.status, baru)
    END IF
    IF baru = DIKIRIM DAN (noResi KOSONG ATAU PANJANG(noResi) < 8) THEN
        THROW ResiTidakValidException
    END IF
    this.status <- baru
```

Tabel transisi ini adalah terjemahan langsung dari diagram status pada SKPL. Dengan bentuk seperti ini, state transition testing bisa disusun dari tabel yang sama, yaitu menguji setiap transisi yang sah dan beberapa transisi yang seharusnya ditolak.

Algoritma #4 Pembatalan Otomatis (Cadangan)

Kelas: PembatalanScheduler | Operasi: batalkanKedaluwarsa(), dijalankan tiap 5 menit

Dalam keadaan normal pembatalan dipicu notifikasi expire dari Midtrans. Penjadwal ini hanya cadangan bila notifikasi tidak pernah sampai. Sebelum membatalkan, status transaksi dicek dulu ke Midtrans supaya transaksi yang ternyata sudah dibayar tidak ikut dibatalkan.

```
@Scheduled(fixedRate = 5 menit)
@Transactional
batalkanKedaluwarsa() -> int
    daftar <- transaksiRepo.findBelumDibayarSebelum(SEKARANG - 15 menit)
    FOR EACH trx IN daftar
```

```
        statusGateway <- midtransService.cekStatus(trx.kode)
        IF statusGateway IN [settlement, capture] THEN
            pesananService.tandaiLunas(trx)
            CONTINUE
        END IF
        FOR EACH pesanan IN trx.daftarPesanan
            FOR EACH item IN pesanan.items
                item.produk.tambahStok(item.jumlah)
            END FOR
            pesanan.ubahStatus(DIBATALKAN)
        END FOR
    END FOR
    RETURN JUMLAH(daftar)
```

Algoritma #5 Verifikasi Notifikasi Midtrans

Kelas: MidtransService | Operasi: proses(notif)

Signature dihitung dari gabungan order_id, status_code, gross_amount, dan server key, lalu di-hash dengan SHA-512. Perbandingan memakai fungsi waktu-konstan supaya tidak bisa ditebak lewat lama proses. Seluruh method berjalan dalam satu transaksi database.

```
@Transactional
proses(notif)
    harapan <- SHA512(notif.order_id + notif.status_code
                      + notif.gross_amount + SERVER_KEY)
    IF NOT samaWaktuKonstan(harapan, notif.signature_key) THEN
        THROW SignatureTidakValidException          // dibalas HTTP 403
    END IF

    trx <- transaksiRepo.findByKodeForUpdate(notif.order_id)
    IF trx KOSONG THEN THROW TransaksiTidakDitemukanException
    IF trx.isFinal() THEN RETURN                    // notifikasi ganda
    IF notif.gross_amount <> trx.totalBayar THEN
        THROW NominalTidakCocokException
    END IF

    SWITCH notif.transaction_status
        CASE settlement, capture:
            trx.pembayaran.perbaruiStatus(LUNAS, notif.payment_type)
            // seluruh pesanan menjadi PERLU_DIPROSES
            pesananService.tandaiLunas(trx)
        CASE expire, cancel, deny:
            trx.pembayaran.perbaruiStatus(GAGAL, notif.payment_type)
            // stok dikembalikan, hanya sekali
            pesananService.batalkan(trx)
        CASE pending:
            // tidak ada perubahan
    END SWITCH
    transaksiRepo.save(trx)
```

Kombinasi status ini bisa langsung dijadikan decision table pada tahap pengujian: signature valid atau tidak, transaksi final atau belum, nominal cocok atau tidak, dan empat kelompok status Midtrans.

Algoritma #6 Atur Ulang Kata Sandi

Kelas: AuthService | Operasi: mintaReset(email), resetPassword(token, baru)

```
mintaReset(email)
    user <- userRepo.findByEmail(email)
    IF user ADA THEN
        token <- acakAman(32 byte)
        user.resetTokenHash    <- SHA256(token)
        user.resetKedaluwarsa  <- SEKARANG + 30 menit
        kirimEmail(user.email, tautan berisi token)
    END IF
    // jawaban selalu sama, ada atau tidak emailnya

resetPassword(token, baru)
    user <- userRepo.findByResetTokenHash(SHA256(token))
    IF user KOSONG ATAU SEKARANG > user.resetKedaluwarsa THEN
        THROW TautanTidakValidException
    END IF
    validasiKataSandi(baru)                        // BR-13
    user.password         <- BCRYPT(baru)
    user.resetTokenHash   <- NULL                   // sekali pakai
    tokenStore.cabutSemua(user.id)                 // sesi lain berakhir
```

Token disimpan dalam bentuk hash, sama seperti kata sandi, supaya kebocoran basis data tidak langsung bisa dipakai untuk mengambil alih akun. Jawaban yang selalu sama mencegah orang lain menebak email mana yang terdaftar.

Algoritma #7 Pembatalan oleh Pembeli

Kelas: PesananService | Operasi: batalkanOlehPembeli(idTrx, idPembeli)

Pembatalan selalu dicek dulu ke Midtrans. Tanpa pengecekan ini, pembeli yang membayar beberapa detik sebelum menekan Batalkan bisa kehilangan pesanannya padahal uangnya sudah masuk.

```
@Transactional
batalkanOlehPembeli(idTrx, idPembeli)
    trx <- transaksiRepo.findByIdForUpdate(idTrx)
    IF trx KOSONG ATAU trx.pembeli.id <> idPembeli THEN
        THROW AksesDitolakException
    END IF
    IF trx.isFinal() THEN
        THROW TidakBisaDibatalkanException          // BR-15
    END IF

    statusGateway <- midtransService.cekStatus(trx.kode)
    IF statusGateway IN [settlement, capture] THEN
        tandaiLunas(trx)
        THROW SudahDibayarException
    END IF
    midtransService.batalkanTransaksi(trx.kode)

    FOR EACH pesanan IN trx.daftarPesanan
        FOR EACH item IN pesanan.items
            item.produk.tambahStok(item.jumlah)
        END FOR
        pesanan.ubahStatus(DIBATALKAN)
```

```
        notifikasiService.kirim(pesanan.toko.pemilik, ...)
    END FOR
    trx.pembayaran.perbaruiStatus(DIBATALKAN, NULL)
    notifikasiService.kirim(idPembeli, ...)
```

Setelah transaksi dibatalkan, Midtrans tetap mengirim notifikasi cancel. Notifikasi itu diabaikan oleh Algoritma #5 karena transaksi sudah final, sehingga stok tidak dikembalikan dua kali.

Query #1 Pencarian Produk dengan Skor

Kelas: ProdukRepository | Operasi: cari(keyword, filter, page)

```
SELECT p.*, t.nama_toko, t.rating,
       ( CASE WHEN p.nama LIKE CONCAT(:kw, '%')        THEN 3
              WHEN p.nama LIKE CONCAT('%', :kw, '%')   THEN 2
              ELSE 1 END ) * 10
       + t.rating * 2
       + LEAST(p.terjual, 100) / 10          AS skor
FROM   produk p JOIN toko t ON p.id_toko = t.id_toko
WHERE  p.status = 'AKTIF' AND t.status = 'AKTIF'
  AND  (p.nama      LIKE CONCAT('%', :kw, '%')
        OR p.deskripsi LIKE CONCAT('%', :kw, '%'))
  AND  (:kategori IS NULL OR p.id_kategori = :kategori)
  AND  p.harga BETWEEN :hargaMin AND :hargaMax
ORDER  BY (p.stok > 0) DESC, skor DESC
LIMIT  20 OFFSET :offset;
```

Produk yang stoknya habis selalu berada di urutan terakhir berapa pun skornya.

Query di atas dipakai untuk urutan Paling Sesuai. Untuk pilihan urutan lain pada FR-28, hanya bagian ORDER BY yang diganti, sedangkan aturan stok habis di urutan terakhir tetap dipertahankan.

Tabel 3.8 Pilihan urutan hasil pencarian

Pilihan Urutan	Bagian ORDER BY
Paling Sesuai	(p.stok > 0) DESC, skor DESC
Terbaru	(p.stok > 0) DESC, p.dibuat DESC
Terlaris	(p.stok > 0) DESC, p.terjual DESC
Harga Termurah	(p.stok > 0) DESC, p.harga ASC
Harga Termahal	(p.stok > 0) DESC, p.harga DESC

Nilai urutan dari klien tidak pernah disisipkan langsung ke query. Server mencocokkannya ke daftar di atas dan memakai Paling Sesuai bila nilainya tidak dikenal, supaya tidak ada celah SQL injection.

Query #2 Laporan Penjualan Toko

Kelas: LaporanService | Operasi: laporanToko(idToko, dari, sampai)

```
SELECT pr.nama,
       SUM(pi.jumlah)                      AS terjual,
       SUM(pi.jumlah * pi.harga_saat_beli) AS pendapatan
FROM   pesanan ps
       JOIN pesanan_item pi ON pi.id_pesanan = ps.id_pesanan
       JOIN produk pr       ON pr.id_produk  = pi.id_produk
WHERE  ps.id_toko = :idToko
  AND  ps.status  = 'SELESAI'
  AND  ps.tgl_selesai BETWEEN :dari AND :sampai
GROUP  BY pr.id_produk, pr.nama
ORDER  BY pendapatan DESC;
```

Pendapatan dihitung dari harga_saat_beli, bukan harga produk saat ini, dan hanya dari pesanan yang sudah selesai karena pesanan yang masih berjalan masih bisa dibatalkan.

Query #3 Produk dengan Stok Menipis

Kelas: ProdukRepository | Operasi: findMenipis(idToko)

```
SELECT p.id_produk, p.nama, p.stok, p.batas_stok,
       CASE WHEN p.stok = 0 THEN 'HABIS' ELSE 'MENIPIS' END AS keterangan
FROM   produk p
WHERE  p.id_toko = :idToko
  AND  p.status  = 'AKTIF'
  AND  p.stok   <= p.batas_stok
ORDER  BY p.stok ASC, p.nama ASC;
```

Batas stok bernilai bawaan 5 dan hanya bisa diisi 1 sampai 100 (BR-17). Notifikasi stok menipis dikirim satu kali saat stok pertama kali menyentuh batas, yaitu di dalam method kurangiStok(), bukan setiap kali query ini dijalankan. Nilai 4, 5, dan 6 untuk batas 5 cocok diuji dengan boundary value analysis.

4. Matriks Kerunutan (Requirement Traceability Matrix)

Tabel berikut memetakan setiap kebutuhan fungsional ke use case dan kelas yang menanganinya, dan nanti dipakai untuk memastikan tidak ada kebutuhan yang terlewat saat pengujian.

Tabel 4.1 Matriks kerunutan kebutuhan

Kode FR	Nama Kebutuhan	Use Case	Kelas yang Menangani
FR-01	Registrasi	Autentikasi	AuthController, AuthService, User
FR-02	Login	Autentikasi	AuthController, AuthService
FR-03	Logout	Autentikasi	AuthController, AuthService
FR-04	Ganti Password	Autentikasi	AuthService, User
FR-05	Lupa Password	Autentikasi	AuthService, User
FR-06	Cari Produk	Cari Produk	ProdukController, ProdukRepository
FR-07	Lihat Detail Produk	Cari Produk	ProdukController, Produk, Ulasan
FR-08	Kelola Keranjang	Kelola Keranjang dan Alamat	KeranjangController, ItemKeranjang
FR-09	Kelola Alamat	Kelola Keranjang dan Alamat	AlamatController, Alamat
FR-10	Checkout	Checkout	CheckoutController, CheckoutService, OngkirCalculator, Transaksi, Pesanan
FR-11	Bayar Pesanan	Bayar Pesanan, Proses Notifikasi Pembayaran	MidtransService, WebhookController, Pembayaran
FR-12	Lacak Pesanan	Lacak dan Konfirmasi Pesanan	PesananController, Pesanan
FR-13	Konfirmasi Pesanan Diterima	Lacak dan Konfirmasi Pesanan	PesananService, PembatalanScheduler
FR-14	Beri Ulasan	Beri Ulasan	UlasanController, Ulasan, Toko
FR-15	Buka Toko	Buka Toko dan Kelola Produk	TokoController, Toko
FR-16	Kelola Produk	Buka Toko dan Kelola Produk	ProdukController, Produk
FR-17	Proses Pesanan	Proses Pesanan	PesananController, PesananService, StatusPesanan
FR-18	Laporan Toko	Laporan Toko	LaporanService
FR-19	Monitoring Transaksi	Fungsi Admin	AdminController, MidtransService
FR-20	Verifikasi Toko	Fungsi Admin	AdminController, Toko
FR-21	Kelola Kategori	Fungsi Admin	AdminController, Kategori
FR-22	Blokir Akun	Fungsi Admin	AdminController, User, Toko

Kode FR	Nama Kebutuhan	Use Case	Kelas yang Menangani
FR-23	Dashboard Admin	Fungsi Admin	AdminController
FR-24	Kelola Profil	Kelola Profil	ProfilController, User
FR-25	Lihat Notifikasi	Lihat Notifikasi	NotifikasiController, NotifikasiService, Notifikasi
FR-26	Lihat Bantuan	Lihat Bantuan	Halaman statis pada aplikasi web dan Android
FR-27	Kunjungi Toko	Cari Produk	TokoController, Toko, ProdukRepository
FR-28	Urutkan Hasil Pencarian	Cari Produk	ProdukController, ProdukRepository
FR-29	Kelola Wishlist	Kelola Wishlist	WishlistController, Wishlist
FR-30	Tulis Catatan untuk Penjual	Checkout	CheckoutService, Pesanan
FR-31	Batalkan Pesanan	Batalkan Pesanan	PesananController, PesananService, MidtransService
FR-32	Unduh Invoice	Lacak dan Konfirmasi Pesanan	PesananController, InvoiceService
FR-33	Kelola Profil Toko	Buka Toko dan Kelola Produk	TokoController, Toko, PembatalanScheduler
FR-34	Balas Ulasan	Beri Ulasan	UlasanController, Ulasan
FR-35	Pantau Stok Menipis	Buka Toko dan Kelola Produk	ProdukController, Produk, ProdukRepository
FR-36	Turunkan Produk	Fungsi Admin	AdminController, Produk, NotifikasiService

Seluruh tiga puluh enam kebutuhan fungsional sudah memiliki use case dan kelas yang menanganinya. Kebutuhan dengan perhitungan atau aturan, yaitu FR-10, FR-11, dan FR-17, menjadi prioritas utama pada tahap pengujian, disusul FR-31 dan FR-35 yang punya aturan batas dan kondisi balapan.